

Proposta de sistema de agendamento de tarefas para *workflow* baseado em prioridades

Igor Steinmacher, Niccholas Rodriguez Vidal, Tiago Lopes Telecken e José Valdeni de Lima
Instituto de Informática - Universidade Federal do Rio Grande do Sul

Resumo— Existe um grande número de pesquisas que buscam mecanismos para aumentar a eficiência do agendamento de tarefas em um sistema de *workflow*. Este aumento de eficiência é alcançada através do desenvolvimento de técnicas que buscam priorizar as tarefas de usuários de forma a reduzir os trabalhos atrasados. Este artigo apresenta uma proposta de arquitetura de um sistema que permita aos atores de um sistema de *workflow* receber suas tarefas, ordenadas de forma a aumentar a eficiência na execução dos processos. Aliados a este sistema existem ainda um módulo responsável pela percepção (*awareness*) dos usuários, mantendo-os informados da urgência das atividades, e das razões pelas quais as tarefas estão ordenadas daquela maneira, uma maneira de antecipar tarefas estritamente sequenciais e um módulo responsável pelo armazenamento (*caching*), dos documentos relacionados às tarefas no cliente, permitindo a realização das tarefas em modo desconectado.

Palavras Chave— Agenda de Tarefas, *awareness*, prioridades, *workflow*

I. INTRODUÇÃO

Gerenciamento de *workflow* se refere a atividades gerenciais manuais ou automáticas, como modelar processos, alocar tarefas a papéis que são assumidos por atores, controlar o andamento das tarefas, monitorar os processos e alertar usuário e gerentes de eventos especiais e excepcionais no sistema [2], [3], [6], [9]. Este trabalho é direcionado a aspectos referentes ao controle do andamento das tarefas e monitoramento de processos, buscando diminuir o atraso na execução das tarefas, explicitando tal fato aos usuários.

Uma das maneiras de acelerar um processo é através da priorização das tarefas envolvidas no *workflow*. Esta priorização é realizada de forma a sugerir aos atores do processo uma ordem de execução das tarefas. Um fator importante é tentar deixar claro ao usuário quais são os parâmetros através dos quais aquela ordem foi dada às tarefas.

Outro ponto que pode ser utilizado para acelerar os processos, é a possibilidade de operar desconectado da rede, mantendo os dados relativos às tarefas prioritárias em um *cache* localizado no cliente.

Afim de colocar em prática os pontos citados acima a arquitetura de um sistema que contenha módulos responsáveis por priorizar as tarefas, manter os usuários informados das razões destas prioridades e possibilitar a execução de tarefas

em modo desconectado é proposta neste trabalho.

O restante do trabalho está dividido da seguinte maneira. A seção II traz os trabalhos que, de alguma forma, estão relacionados ao presente trabalho. A seção III traz a da arquitetura do sistema proposto. A seção IV apresenta o andamento da implementação do protótipo do sistema e a seção V traz as conclusões e trabalhos futuros relacionados à proposta.

II. TRABALHOS RELACIONADOS

Mecanismos de priorização de tarefas para *workflow* tem sido alvo de muitos trabalhos e pesquisas [3], [8], [10], [13] recentemente. Estas pesquisas visam reduzir o atraso dos processos.

Cada ator do *workflow* possui uma fila de tarefas. Estas tarefas são ordenadas de acordo com uma certa prioridade. As políticas de priorização mais conhecidas e utilizadas são apresentadas a seguir:

- **FCFS** (*First-Come-First-Serve*) ou FIFO: A grande vantagem deste método é a simplicidade, e é muito útil quando o grafo do *workflow* é simples. Muitas das abordagens existentes são variação desta política.
- **SPT** (*Shortest Processing Time*): chamada por alguns autores de SWT (*Shortest Work-Time*). Este método prioriza as tarefas com os menores tempos de processamento. Apesar de aumentar o número de tarefas concluídas e aliviar o risco de aumento excessivo da fila de tarefas, esta política pode atrasar muito processos que tenham tarefas com grandes tempos de duração.
- **LPT** (*Largest Processing Time*): chamada por alguns autores de LWT (*Largest Work-Time*). Ao contrário do SPT, esta política prioriza as tarefas que possuem maior tempo de duração estimado. Este tipo de método acaba por atrasar muito as tarefas pequenas, causando atraso médio muito grande para alguns processos.
- **EDD** (*Earliest Due Date*): Prioriza as tarefas baseada na data estimada para a finalização da tarefa. A tarefa com a menor diferença entre o data de finalização estimada e o tempo atual tem a maior prioridade.

Tramontina [10] aplica técnicas e conceitos de agendamento (*scheduling*) ao *workflow* e aponta algumas

dificuldades encontradas em utilizar este tipo de técnica, incluindo: incertezas nos tempos de execução das tarefas, a natureza dinâmica do agendamento das tarefas no *workflow* e a escolha do critério de otimização. Uma abordagem para tentar resolver as incertezas é proposta, utilizando um algoritmo genético para resolver o problema de agendamento, levando em conta as rotas e os tempos de execução.

Outra proposta de algoritmo genético para agendamento de trabalhos é encontrada em [8]. Este apresenta resultados muito bons, porém a abordagem é proposta para uma linha de produção, com máquinas, sem necessidade de propor uma ordem, o que ocorre é a imposição de uma ordem de prioridades.

Em Zhao [3] é apresentado um *framework* de redução de tempo de execução dos processos em um sistema de gerenciamento de reclamações. Isto é feito através do reordenamento da lista de tarefas. A priorização das tarefas é feita de acordo com a política de recompensas utilizada. As políticas citadas no trabalho são: (i) Número de reclamações completadas por unidade de tempo; (ii) número de tarefas cumpridas por unidade de tempo e (iii) “valor de negócio” enviado por unidade de tempo.

Quanto ao armazenamento de documentos junto ao cliente, [5] propõe uma arquitetura hierárquica de *caches* para manter documentos próximos ao usuário visando diminuir a latência sentida pelo usuário. Nesta arquitetura existem dois níveis, em um nível estão os documentos compartilhados por um grupo de trabalho, e no outro nível se encontram os documentos referentes a um único cliente. A proposta deste trabalho possui uma política que prevê a utilização de uma agenda de tarefas como um de seus parâmetros, recuperando documentos que podem ser necessários em breve, de acordo com as próximas tarefas a serem realizadas pelo usuário. Essa proposta, é uma das primeiras que busca unir pré-busca com uma abordagem baseada em tarefas, porém, apresenta apenas um sistema que coloque os documentos num *cache* do usuário sem se preocupar com prioridades das tarefas, apenas ordenando os documentos de acordo com recomendações e histórico de acessos.

III. ARQUITETURA PROPOSTA

A arquitetura proposta é apresentada na Figura 1, e possui alguns módulos no lado servidor e um módulo de gerenciamento de *cache* no lado cliente. A seguir serão detalhados cada um dos módulos.

A. Servidor

No lado servidor se encontram o gerenciador de prioridades e *caching* além do módulo gerenciador de *awareness*. Estes se relacionam entre si e com a máquina de *workflow*.

1) Gerenciador de Prioridades e Caching

A principal função deste módulo é recolher as informações disponibilizadas pela máquina de *workflow* e ordená-las segundo suas prioridades. Esse ordenamento é feito seguindo uma função que visa priorizar aquelas tarefas que estão, de alguma maneira, servindo de gargalo para o processo e que estão próximas de seu prazo final. Numa primeira implementação a função de prioridades (1) apresentou o seguinte formato:

$$P_{ij} = \left(\frac{Ta_{ij}}{T_j} + \frac{Tpc_{ij}}{Tp_{ij}} \right) * \frac{td_{ij}}{t_{ij}} \quad (1)$$

Onde Ta_{ij} representa o número de tarefas que vem após a tarefa i da instância j , isto é, quantas tarefas estão aguardando a conclusão daquela tarefa para serem iniciadas. T_j representa o número total de tarefas na instância j do processo, Tpc_{ij} é o número de tarefas paralelas à tarefa i da instância j que já foram concluídas, enquanto Tp_{ij} é o número total de tarefas paralelas àquela. Na continuação, td_{ij} representa o tempo decorrido desde quando a tarefa i da instância j (ti_{ij} da Equação 2) foi iniciada até o presente momento, e t_{ij} representa o tempo médio para a conclusão da tarefa i , este valor é obtido através de informações de duração máxima (d_{max}) e mínima (d_{min}) de cada tarefa estimadas quando da modelagem do processo.

$$td_{ij} = t_{atual} - ti_{ij} \quad (2)$$

$$t_{ij} = \frac{(d_{max} + d_{min})}{2} \quad (3)$$

Com esta equação (1) se pretende priorizar primeiramente aquelas tarefas que disparam um grande número de outras tarefas. Prioriza-se também aquelas tarefas que estão servindo de gargalo, estando em paralelo com outras tarefas que já foram concluídas.

Esta é apenas uma proposta inicial de função criada para possibilitar a continuidade da implementação dos outros módulos, e está sendo estudada e alterada a fim de trazer melhores resultados. Esta função foi submetida a testes preliminares de simulação e apresentou tempo de atraso médio menor do que utilizando-se métodos como o FIFO, SPT e LPT. É válido salientar que tais testes foram realizados com várias instâncias de modelo de *workflow* simples. Quando aplicada a instâncias de modelos diferentes e de *workflows* mais elaborados, os resultados podem variar. Outras simulações de maior porte estão sendo idealizadas e serão realizadas afim de aprimorar a função e compará-la a outros métodos encontrados na literatura.

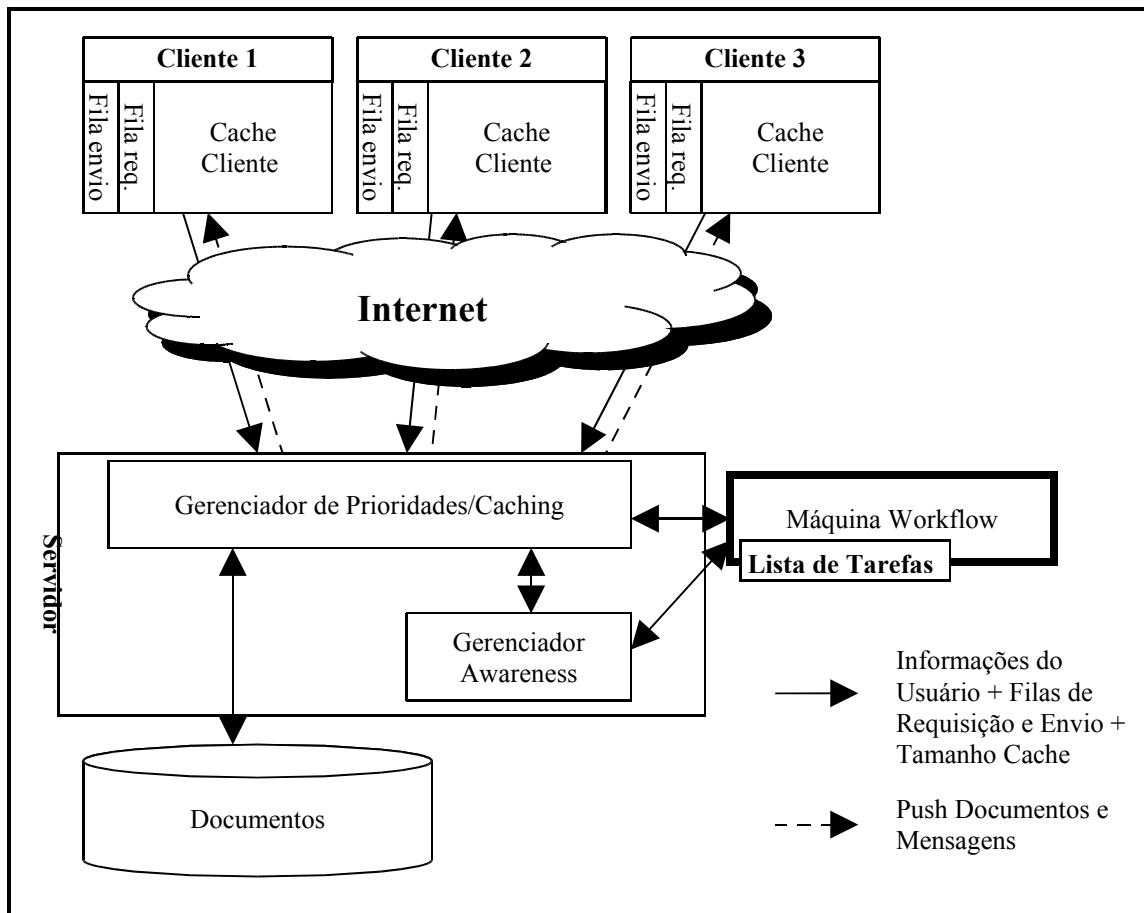


Fig. 1. Arquitetura proposta

A ordenação das tarefas fornecida através desta função será apresentada apenas como uma sugestão ao usuário, podendo este optar pela execução das tarefas na ordem que lhe for mais interessante. Os métodos apresentados na seção II serão outras opções apresentadas.

Os resultados desta do mecanismo escolhido servirão de entrada para o mecanismo de *caching*, que enviará os documentos para o usuário segundo a ordem de prioridades dos mesmo. Isto é necessário visto que o usuário poderá restringir o tamanho de seu *cache* local, recebendo apenas os documentos das tarefas mais urgentes. Estes documentos serão enviados pelo servidor através da técnica de *push*, assim que o servidor for avisado da conexão do cliente. Este mecanismo de *caching* funcionará apenas nos casos em que o usuário indique que trabalha em modo desconectado, ficando isto armazenado em seu perfil, podendo ser modificado a qualquer momento.

Ainda neste módulo está previsto o mecanismo responsável por controlar as solicitações de antecipação de tarefas. Esta antecipação acontecerá quando um usuário desejar realizar uma tarefa que ainda não foi disparada. Por exemplo, se um usuário não possui nenhuma tarefa liberada em sua agenda, é possível que este solicite que uma das tarefas que está aguardando ser disparada seja liberada. O mecanismo verificará as pendências dessa tarefa e solicita aos usuários

que estão realizando as tarefas imediatamente anteriores àquela, que mandem seus resultados parciais ou que liberem os documentos para que uma próxima tarefa possa ser iniciada.

Tal antecipação flexibiliza a execução dos processos de *workflow* através do relaxamento das condições de disparo das tarefas, permitindo que estas possam ser iniciadas antes mesmo de suas tarefas antecessoras estarem totalmente concluídos. Isto pode trazer grandes ganhos ao processo, uma vez que isto provê um aumento no paralelismo entre tarefas estritamente sequenciais.

2) Gerenciador de Awareness

Este módulo é de grande importância dentro deste sistema pois é o responsável por manter o usuário ciente do que está acontecendo com relação à suas tarefas e aos processos como um todo.

Uma das funcionalidades deste módulo é a verificação das razões pelas quais uma tarefa está sendo priorizada e repassar esta informação ao usuário. Da mesma maneira ela é responsável por verificar a causa da não liberação das tarefas e apresenta-las ao usuário.

Outra funcionalidade deste módulo diz respeito à antecipação das tarefas. Quando ocorre uma solicitação de antecipação, o gerenciador de prioridades repassará as

informações ao gerenciador de *awareness* que enviará mensagens aos responsáveis pela conclusão das tarefas, que podem ou não liberar seus documentos. Quando a resposta é enviada, este gerenciador deverá repassar uma mensagem ao usuário que requisitou o adiantamento, liberando ou não a tarefa. A proposta deste módulo visa diminuir a comunicação entre diferentes atores dentro do processo, deixando que estes se comuniquem com o sistema.

Algumas outras funções deste módulo estão sendo estudadas, de forma a tentar manter o usuário com o máximo de ciência do processo como um todo.

B. Cliente

A interface do usuário com o sistema se dará através de páginas HTML geradas dinamicamente no servidor, através de PHP, e poderão ser acessadas por qualquer navegador web. Como dito anteriormente, no lado cliente poderão ser mantidos alguns documentos relacionados às tarefas atribuídas àquele usuário e que já tenham sido disparadas. Estes documentos são armazenados de acordo com o tamanho disponibilizado para armazenamento.

Quando da utilização do *cache* para possibilitar a operação em modo desconectado, serão utilizadas duas filas junto ao *cache*. Uma fila de requisições não atendidas (*cache miss*) e outra com dados das solicitações de envio (*uploads, feedback* do andamento das tarefas, mensagem de solicitação de adiantamento de tarefas). Estas filas são processadas quando houver uma nova conexão com o servidor.

A interface da agenda de tarefas do cliente é apresentada na Figura 2 e apresenta uma tabela contendo alguns campos, explicados a seguir:

- *Tarefa*: este campo traz o nome da tarefa, que é um link para informações avançadas da tarefa (se houverem);
- *Documentos*: neste campo se encontram *links* para os documentos que serão utilizados naquela tarefa;
- *Data Início*: Data em que a tarefa foi disparada;
- *Data Fim*: Data prevista para o fim da tarefa (informação fornecida na definição dos processos);
- *Urgência*: Nível de urgência daquela tarefa. Este nível é dado por valores entre 1 e 5 (sendo 1 atribuído às tarefas de mínima urgência e 5 às tarefas de máxima urgência). Estes níveis de urgência são definidos pelo sistema segundo limites estabelecidos de acordo com o resultado da função de prioridade escolhida. Estes valores são *links* que levam a uma tela que mostra as razões pelas quais aquele trabalho tem aquela prioridade (quantas tarefas aguardando por aquela para serem liberadas, quantas tarefas em paralelo finalizadas, quanto tempo para o *deadline*, ...);
- *Status*: Neste campo existirá um círculo indicando o estado daquela tarefa (verde, amarelo ou vermelho). O círculo for *verde* indica que a tarefa está disponível com todos seus documentos disponíveis e sem

pendências. Se o círculo for *amarelo* significa que a tarefa está liberada, porém com pendências (por exemplo, utilizando resultados intermediários de fornecidos por tarefas antecessoras devido a solicitação de antecipação ou a opção de modo desconectado está ativa e não foi possível armazenar todos os documentos relativos àquela tarefa no *cache* local). Estes indicadores também são *links* que levam a explicações das causas de cada uma das cores. No caso do indicador vermelho, aparecerá uma opção de “Solicitar Antecipação”;

- *Feedback*: Através deste link o usuário poderá enviar ao sistema *feedback* do andamento de sua atividade, podendo dar como concluída uma tarefa ou indicar como está o andamento da tarefa. Também é possível enviar resultados intermediários da tarefa.

As tarefas são apresentadas nessa tabela em ordem decrescente de prioridade, apenas sugerindo uma ordem de execução para o usuário. Como em um sistema de *workflow*, existe muita incerteza, uma otimização exata não é possível, pois humanos não podem ser programados com máquinas controladas numericamente [1], [3]. Torna-se necessário que o usuário tome conhecimento da importância de realizar a tarefa naquela ordem. Por esse motivo o sistema ajuda o usuário oferecendo explicações sobre o nível de urgência e o estado em que se encontra aquela tarefa.

Tarefa	Documentos	Data Início	Data Limite	Urgência	Status	Feedback
Tarefa3	Documento2 Documento4	25/03/2004	20/06/2004	3		Enviar
Tarefa5	Documento2 Documento5 Documento7	20/03/2004	05/05/2004	2		Enviar
Tarefa4	Documento4		30/11/2004	1		Enviar
Tarefa6	Documento2 Documento4 Documento7		15/07/2004	1		Enviar
Tarefa7	Documento2 Documento4		25/08/2004	1		Enviar
Tarefa8	Documento4		10/12/2004	1		Enviar

Fig 2. Interface do usuário contendo sua agenda de tarefas

1) Gerenciador de Caching Cliente

O objetivo do gerenciador de *caching* cliente é manter os documentos necessários para a realização das tarefas do agenda de tarefas sempre acessíveis para o usuário, tornando transparente para ele o fator conectado/desconectado.

O gerenciador de *caching* cliente é um *proxy* do tipo local, que pode ser acessado e configurado através de um navegador *Web*. Desta forma, o dispositivo cliente pode funcionar de forma autônoma.

O gerenciador de *caching* possui três módulos essenciais: (i) o *scheduler*; (ii) o gerenciador de documentos e (iii) o monitor da rede.

O *scheduler* será responsável pelos *downloads* e *uploads* de documentos. Se o usuário tentar acessar um documento que não está na cache (*miss*), esta requisição será colocada na fila de requisições. Já a fila de envios deverá conter documentos submetidos pelo usuário.

O gerenciador de documentos será responsável por manter todos os *downloads* que foram completados com sucesso. O tamanho dos documentos e a importância da tarefa associada aos documentos serão levados em consideração quando o espaço para armazenamento for restrito.

O monitor da rede deverá se comunicar com o sistema operacional do cliente e informar ao *scheduler* as condições da rede. O *scheduler*, uma vez ciente das condições da rede, pode decidir se deve ou não buscar/atualizar documentos.

Se o dispositivo estiver conectado diretamente à rede, o gerenciador de *caching* cliente aproveita esta oportunidade para buscar os documentos que tenham uma alta probabilidade de serem utilizados pelo usuário com base no *worklist* deste usuário.

Porém, se o dispositivo estiver conectado de forma indireta a rede (*wireless*), i.e. com limitações de banda e de energia, o gerenciador de *caching* cliente pode optar por buscar/atualizar somente os documentos que consideram muito relevantes (geralmente aqueles no topo da lista apresentada pelo gerenciador de prioridades). Assim, é possível minimizar o consumo de energia e de banda por parte do dispositivo permitindo que os documentos para a realização de tarefas urgentes trafeguem de maneira rápida.

IV. ESTADO ATUAL DO SISTEMA

Um protótipo contendo todos os módulos citados na seção anterior está sendo implementado. Este protótipo está sendo implementado como parte do projeto CEMT (Concepção de um Ambiente de Edição Cooperativa Multimídia com Tecnologia de *Workflow*) [7]. Neste projeto foram implementadas uma ferramenta de definição de processos [12] e um protótipo de uma máquina de *workflow* [11]. Estas duas implementações estão integradas e em funcionamento, respeitando o modelo sugerido pela WfMC [4].

Seguindo a proposta do projeto, a implementação está sendo realizada utilizando software livre. Alguns módulos já estão prototipados e em funcionamento. O módulo que calcula as prioridades das tarefas está implementada em PHP, conectado a um banco de dados MySQL, uma vez que a máquina de *workflow* implementada utiliza este banco para persistir seus dados. Este módulo utiliza a função apresentada neste artigo, e está em fase de testes. É válido lembrar que a implementação deste módulo foi essencial para que os outros módulos pudessem utilizar sua saída como parâmetro.

O módulo responsável pelo *caching* dos documentos necessários à execução das tarefas em modo desconectado começou a ser estudado, e é um dos próximos passos dentro deste trabalho.

O módulo de gerenciamento de *awareness* está parcialmente implementado, já levando ao usuário uma série

de informações relevantes. Informações sobre razões pelas quais uma tarefa tem determinado nível de urgência e status já estão em funcionamento.

V. CONCLUSÕES E TRABALHOS FUTUROS

O presente trabalho apresentou uma proposta de arquitetura para um sistema que visa priorizar tarefas deixando o usuário ciente da importância de tal ordem. Tal área de pesquisa se mostra muito promissora, apesar de necessitar maiores esforços, visto a grande complexidade envolvida.

Uma contribuição deste trabalho é a proposta de uma função de priorização de tarefas que tem como parâmetros informações do modelo do processo. Foram realizadas simulações utilizando algumas técnicas de avaliação de performance encontradas em [8], [10], porém aplicadas à poucas instâncias de um mesmo processo. Os resultados mostraram ganhos em termos de tempo médio de atraso se comparado ao FIFO, SPT e LPT. Simulações com um maior número de instância de diferentes processos com diferentes durações estão sendo idealizadas, afim de aprimorar-se a função, reduzindo ainda mais os tempos de atraso.

Outra contribuição é a maneira com que as tarefas são mostradas ao usuário, que pode perceber a importância da execução das tarefas naquela ordem e as razões pelas quais uma tarefa não está liberada, oferecendo oportunidade do usuário solicitar a antecipação da mesma, esperando-se uma diminuição no tempo ociosos dos atores e no tempo médio de atraso. Tal antecipação visa tornar possível que tarefas estritamente seqüências sejam executadas, em parte, de maneira paralela. Tal mecanismo visa reduzir o tempo de execução dos processos.

Como trabalhos futuros estão a implementação do sistema que faça o *push* dos documentos para junto do cliente, bem como o sistema de gerenciamento das filas de envio e requisição. Isto permitirá aos usuários executar suas tarefas mesmo em modo desconectado. Ao fim desta implementação, pretende-se obter um sistema de *workflow* com um módulo de agendamento de tarefas que permita aos usuários executarem suas tarefas de forma a diminuir o atraso através da percepção do processo, da possibilidade de trabalhar desconectado e de poder solicitar a antecipação de tarefas.

VI. AGRADECIMENTOS

O autores deste trabalho são gratos pelo financiamento por parte do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) e da Coordenação de Aperfeiçoamento de Pessoal de nível Superior (CAPES).

Agradecimentos especiais ao grupo SIGHA, do qual faz parte o projeto CEMT, pelo apoio, dicas e pelo ambiente de trabalho oferecido.

VII. REFERÊNCIAS

- [1] O. Berman , R.C. Larson , E.Pinker, “Scheduling workforce and *workflow* in a high volume factory”, *Management Science*, v.43 n.2, p.158-172, Feb. 1997
- [2] J.L. Zhao, A. Kumar, “Data Management Issues for Large Scale, Distributed *Workflow* Systems on the Internet”, *ACM SIGMIS Database*, v.29, pp. 22-32, Fall,1999.
- [3] J.L., Zhao and E.A. Stohr, “Temporal *Workflow* “Management in a Claim Handling System”. SIGSOFT: Software Engineering Notes, v. 24, pp. 187-95, Mar, 1999.
- [4] WfMC, “*Workflow* reference model.” Technical report, *Workflow* Management Coalition, Brussels, 1994.
- [5] J.Kaiser, C. Vaill and S. Dossick, “A workgroup model for smart pushing and pulling”, In *Proc. IEEE 8th International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises*, Stanford, 1999. pp 15-21.
- [6] G. Kappel, S. Rausch-Schott and W. Retschitzegger, “Coordination in *workflow* management systems - a rule-based approach.” In *proc. Coordination Technology for Collaborative Applications: Organization, Processes and Agents*. Springer-Verlag, 1998. pp. 99-119.
- [7] J. Lima, V. Quint, N. Layaida, N. Edelweiss, C. Zeve, M. Pinheiro, T.L. Telecken, “The Conception of Cooperative Environment for Editing Multimedia Documents With *Workflow* Technology”. In *proc. Protem CC 2001 – Projects Evaluation Workshop – NSF/INRIA*. Rio de Janeiro – Brazil. p. 542-560.
- [8] D.C. Mattfeld, C. Bierwirth, “Minimizing Job Tardiness: Priority Rules vs. Adaptive Scheduling”, In *proc. International Conference on Adaptive Computing in Design and Manufacture*, Londres, 1998. pp. 59-67.
- [9] E.A. Stohr and J. L. Zhao, “A technology adaptation model for business process automation”, In *proc. 30th Annual Hawaii International Conference on Systems Sciences*, January, 1997.
- [10] G.B. Tramontina, J. Wainer and C. Ellis, “Applying Scheduling Techniques to Minimize the Number of Late Jobs in *Workflow* Systems”, In *proc 2004 ACM symposium on Applied computing*, 2004. pp. 1396-1403
- [11] M.B. França, “Proposta e implementação de uma máquina de *workflow* para o projeto CEMT”, Dissertação (Mestrado em Ciência da Computação) - Instituto de Informática, UFRGS, Porto Alegre, 2004. 97f.
- [12] T.L. Telecken “Um estudo sobre modelos conceituais para ferramentas de definição de processos de *workflow*” Dissertação (Mestrado em Ciência da Computação) - Instituto de Informática, UFRGS, Porto Alegre, 2004. 123f.
- [13] I. Steinmacher, “Técnicas de Web Caching e Prefetching com Prioridades” Trabalho Individual – Instituto de Informática, UFRGS, Porto Alegre, 2004.

VIII. BIOGRAFIAS



Igor Steinmacher nasceu em Maringá, Paraná em 22 de Março de 1980. Bacharel em Ciência da Computação pela Universidade Estadual de Maringá. Atualmente cursando Mestrado em Ciência da Computação na Universidade Federal do Rio Grande do Sul, sob orientação do Prof. Dr. José Valdeni de Lima. Sua pesquisa está relacionada a Antecipação de tarefas em sistemas de *Workflow*, *Awareness* e CSCW.



Niccholas Rodriguez Vidal nasceu em Jaboticabal, São Paulo em 21 de Julho de 1980. Engenheiro da Computação pela Universidade Federal do Rio Grande do Sul. Atualmente cursando Mestrado em Ciência da Computação na Universidade Federal do Rio Grande do Sul, sob orientação do Prof. Dr. José Valdeni de Lima. Sua pesquisa está voltada para Adaptatividade, Recuperação de Informação e Autoria.



Tiago Lopes Telecken nasceu em Passo Fundo, Rio Grande do Sul em 24 de Junho de 1975. Bacharel em Ciência da Computação pela Universidade de Passo Fundo e Mestre em Computação pela Universidade Federal do Rio Grande do Sul. Atualmente cursando Doutorado em Ciência da Computação na Universidade Federal do Rio Grande do Sul, sob orientação do Prof. Dr. José Valdeni de Lima. Sua pesquisa está voltada para Workflow, CSCW e Linguagens Visuais.



José Valdeni de Lima nasceu em Fortaleza, Ceará em 01 de Agosto de 1955. Doutor em Informática pela Université Joseph Fourier (França). Professor Assistente do Instituto de Informática da Universidade Federal do Rio Grande do Sul. Atualmente tem como algumas de suas frentes de pesquisa *Workflow*, CSCW, Educação a Distância e Adaptatividade.